Options Pricing & Risk Engine — Project Plan

Oct 2, 2026 · @Nat

Goals and definition of done

Build a C++20 engine that prices a book of about 100,000 options and recomputes every Greek on each tick of the underlying, with measured, published latency numbers. Plan for about 5 weeks of evenings and weekends.

The project is done when it has:

- **Pricers:** closed-form Black-Scholes for European options, a binomial tree for American options, and a Monte Carlo pricer used only as an independent check.
- **Implied volatility:** a solver that turns a market price back into a volatility, robust at the edges (deep in or out of the money, near expiry).
- **Risk:** per-option Greeks (delta, gamma, vega, theta, rho), portfolio totals, and a scenario grid of spot and volatility shocks.
- **A tick-driven loop:** a replayed stream of underlying prices, with the whole book revalued on every tick.
- **A test suite** that proves correctness against a slow, obviously right reference implementation.
- **A write-up** with real numbers: prices per second per core (scalar, SIMD, multithreaded), full-book revalue latency at p50, p99 and p99.9, and implied-vol solves per second.

Speed without proof of correctness is worth little to these firms, so the tests are a first-class deliverable, not cleanup.

Finance primer

You need about five ideas to build this, and all of them come up in quant interviews too.

Options. A call is the right to buy the underlying at the strike price K by expiry T; a put is the right to sell. European options can only be exercised at expiry; American options any time before. Inputs to every pricer: spot S, strike K, time to expiry T (years), risk-free rate r, dividend yield q, and volatility σ.

Black-Scholes. The closed-form price of a European call (the put follows the same pattern, or from put-call parity):

```
C = S e^{-qT} N(d_1) - K e^{-rT} N(d_2), \quad d_1 = \frac{\ln(S/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}
```

N is the standard normal CDF. Computing N fast and accurately is the heart of the performance work.

Put-call parity. For European options, C − P = S e^(−qT) − K e^(−rT), regardless of the model. It is your cheapest correctness check.

The Greeks are the price's sensitivities, and they are what a trading desk actually uses to manage risk:

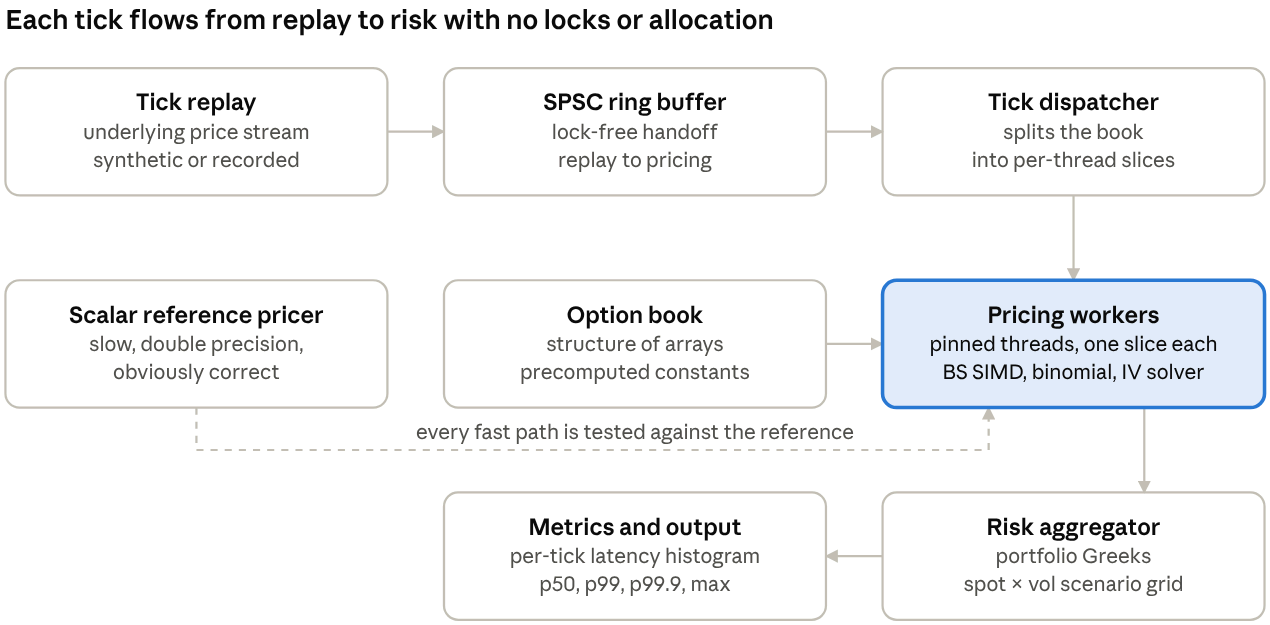
Greek	Sensitivity of price to	Why a desk cares
Delta	Spot	How much underlying to hold to hedge
Gamma	Spot, second order	How fast the hedge goes stale
Vega	Volatility	Exposure to vol moves
Theta	Time	Value lost per day
Rho	Interest rate	Usually small; complete the set

Implied volatility. Markets quote prices, and traders think in volatility. Implied vol is the σ that makes Black-Scholes match a market price. There is no closed form, so you solve numerically: Newton's method using vega, with a bracketing fallback (Brent or bisection) where vega is tiny. Jäckel's "Let's Be Rational" method is the state-of-the-art reference to compare against.

Binomial trees for American options. The Cox-Ross-Rubinstein tree steps the price up by u = e^(σ√Δt) or down by 1/u each step, then works backward from expiry, taking the larger of holding value and exercise value at each node. Accuracy improves roughly with 1/N steps but oscillates between odd and even N; averaging N and N+1, or the Black-Scholes-smoothed variant, fixes most of that.

Architecture

One replay thread feeds pinned pricing threads through a lock-free queue, and a slow scalar pricer serves as the test oracle for every fast path.



engine architecture · tick path and test oracle

Key design choices:

- **Structure-of-arrays book:** spot-independent values (VT, discount factors, $\log K$) are computed once at load, so each tick only redoes the spot-dependent math.
- **No allocation, locks, or system calls on the tick path:** the same discipline as an audio callback.
- **Static partitioning:** each worker owns a contiguous slice of the book, so threads never share cache lines and need no synchronization until aggregation.
- **Swappable kernels** behind one interface, so scalar, SIMD, and reference versions run on identical inputs for both tests and benchmarks.

Week-by-week plan

The order is correct first, then fast, then real-time: each week ends with something that runs and is tested.

Week 1 — Environment and a correct scalar pricer

- [ ] Set up a bare-metal Linux box (dual boot or a spare machine; WSL2 and cloud VMs hide hardware perf counters)
- [ ] CMake project with GoogleTest, Google Benchmark, and ASan/UBSan build presets
- [ ] Scalar double-precision Black-Scholes price and all five Greeks, written for clarity, not speed: this becomes the reference
- [ ] Tests: textbook values, put-call parity, price bounds, Greeks against bump-and-reprice finite differences

Week 2 — Implied vol, American options, independent checks

- [ ] Implied-vol solver: Newton with a bracketed fallback; round-trip test price → vol → price over a grid of moneyness and expiry
- [ ] CRR binomial pricer for American options, with a convergence study (error vs. steps) plotted
- [ ] Monte Carlo European pricer as a third, independent check
- [ ] Cross-check a sample of results against QuantLib

Week 3 — Make it fast

- [ ] Switch the book to structure-of-arrays layout; precompute per-option constants ($\sigma\sqrt{T}$, discount factors, log K)
- [ ] Vectorize the Black-Scholes kernel (AVX2, via intrinsics or a library such as Highway or xsimd)
- [ ] Write a vectorized normal CDF; measure its error against the scalar reference and pick the accuracy/speed trade-off deliberately
- [ ] Benchmark each step separately so the write-up can show where every gain came from

Week 4 — Real-time engine

- [ ] Tick replay: a synthetic book (strikes around spot, expiries from 1 week to 2 years, a simple volatility smile) and a stream of underlying prices
- [ ] Lock-free single-producer/single-consumer ring buffer from replay to pricing
- [ ] Thread pool with pinned threads, each owning a contiguous slice of the book; no allocation or locks on the tick path
- [ ] Portfolio aggregation and a spot × vol scenario grid per tick
- [ ] Per-tick latency histogram (HdrHistogram) with p50, p99, p99.9, and max

Week 5 — Profile, polish, write up

- [ ] Profile with perf stat and flame graphs; fix the top two findings
- [ ] Clean README with build steps, architecture, and the results table
- [ ] Write-up / blog post (see below)

Correctness and testing

Every fast path is tested against the slow scalar reference, and the reference is tested against finance identities that hold regardless of implementation.

- **Identities and bounds:** put-call parity; a call is worth at least max(S e^(−qT) − K e^(−rT), 0); call prices fall as strike rises and are convex in strike.
- **Greeks:** each analytic Greek matches bump-and-reprice finite differences within a stated tolerance.
- **American vs. European:** an American put is worth at least the European put; an American call on a non-dividend stock equals the European call.
- **Implied-vol round trips** over a dense grid, including the hard corners (deep in or out of the money, a day to expiry). Document where the solver fails instead of hiding it.
- **SIMD vs. scalar:** the maximum absolute and relative error over millions of random inputs, reported in the write-up.
- **Property-based tests** (RapidCheck) generate random valid option parameters and check all of the above.
- **Sanitizers in CI:** ASan and UBSan on every build, TSan on the threaded engine.

Performance and benchmarking

The write-up should show a ladder of optimizations, each measured on its own, so a reader can see exactly what every change bought.

Step	What changes	Measure
Baseline	Scalar double, array of structs	Prices/sec, 1 core
Layout	Structure of arrays, precomputed constants	Prices/sec, cache misses
Precision	float where the error budget allows	Prices/sec, max error
SIMD	Vectorized kernel and normal CDF	Prices/sec, IPC
Threads	Pinned threads, one book slice each	Full-book revalue latency

Step	What changes	Measure
Comparison	QuantLib on the same inputs	Prices/sec

Measurement hygiene. Use bare-metal Linux. Isolate cores (`isolcpus`), pin threads, fix the CPU frequency or record it, and warm up before measuring. Report distributions, not averages: p50, p99, p99.9, and max.

Tools to learn along the way:

- `perf stat` for instructions per cycle, cache misses, and branch misses
- `perf record` plus flame graphs to see where time goes
- Compiler vectorization reports (`-Rpass=loop-vectorize` in Clang, `-fopt-info-vec` in GCC) to confirm loops actually vectorized
- `llvm-mca` to reason about the inner loop's throughput
- Compiler Explorer (godbolt) to read the generated assembly

This is the section that also closes the Linux gap: by week 5 you will have used the same tools trading-firm engineers use daily.

Write-up and interview talking points

The write-up is what a recruiter or interviewer will actually read, so treat it as the product.

Structure of the post:

1. What the engine does, in two sentences, with the headline numbers
2. Architecture diagram and the tick path
3. How correctness was proven
4. The optimization ladder, with a chart per step
5. What didn't work, and why (the most interesting part to these readers)
6. What you would do next

Talking points to be ready for:

- Why structure of arrays beat array of structs here, in terms of cache lines and SIMD lanes
- Why the American tree resists vectorization (the early-exercise branch) and what you tried
- Where the implied-vol solver breaks and how you made it robust

- Your float vs. double decision and the error budget behind it
- Why p99.9 matters more than the mean on a trading desk, and how that maps to an audio callback missing its buffer deadline
- Put-call parity and delta hedging intuition: expect these as interview questions on their own

Stretch goals and the path to the mini exchange

Pick at most one stretch goal; the mini exchange is the bigger payoff.

- **Volatility surface fitting:** fit a parametric smile (such as SVI) to quoted prices each tick, instead of using a fixed synthetic smile.
- **Python bindings** with pybind11, so the engine can be driven from a notebook; HRT and most firms pair C++ with Python.
- **An OCaml port** of the reference pricer, if you're aiming at Jane Street and curious about the language.
- **A GPU kernel** for Monte Carlo pricing, as a comparison point.

What carries over to the mini exchange: the lock-free ring buffer, the thread-pinning setup, the latency-histogram harness, and the benchmark discipline all transfer directly. Later, the market-making strategy in the exchange project could use this engine to quote options on the simulated underlying, which ties the two projects into one story.

Resources

- John Hull, *Options, Futures, and Other Derivatives* — the standard reference for the pricing math and worked examples to test against
- Sheldon Natenberg, *Option Volatility and Pricing* — trader intuition for the Greeks and volatility
- Peter Jäckel, "Let's Be Rational" — the reference implied-volatility method
- QuantLib — open-source library to cross-check prices and benchmark against
- Agner Fog's optimization manuals and the Intel Intrinsics Guide — for the SIMD work
- Brendan Gregg's material on perf and flame graphs — for profiling on Linux
- Google Benchmark, HdrHistogram, RapidCheck, Highway or xsimd — the libraries named in the plan